

飛行手順説明書

2026年7月12日 20:24

飛行経路別バッテリー消費電力測定 説明資料

全自動フライト + 区間別バッテリー消費量計測スクリプト

作成 21th T.Uno

WayPointファイルで作成した、上昇、下降、南、西、北、東の四角形の飛行ミッションを自動で実行し、各区間ごとのバッテリー消費量(mAh)を計測する

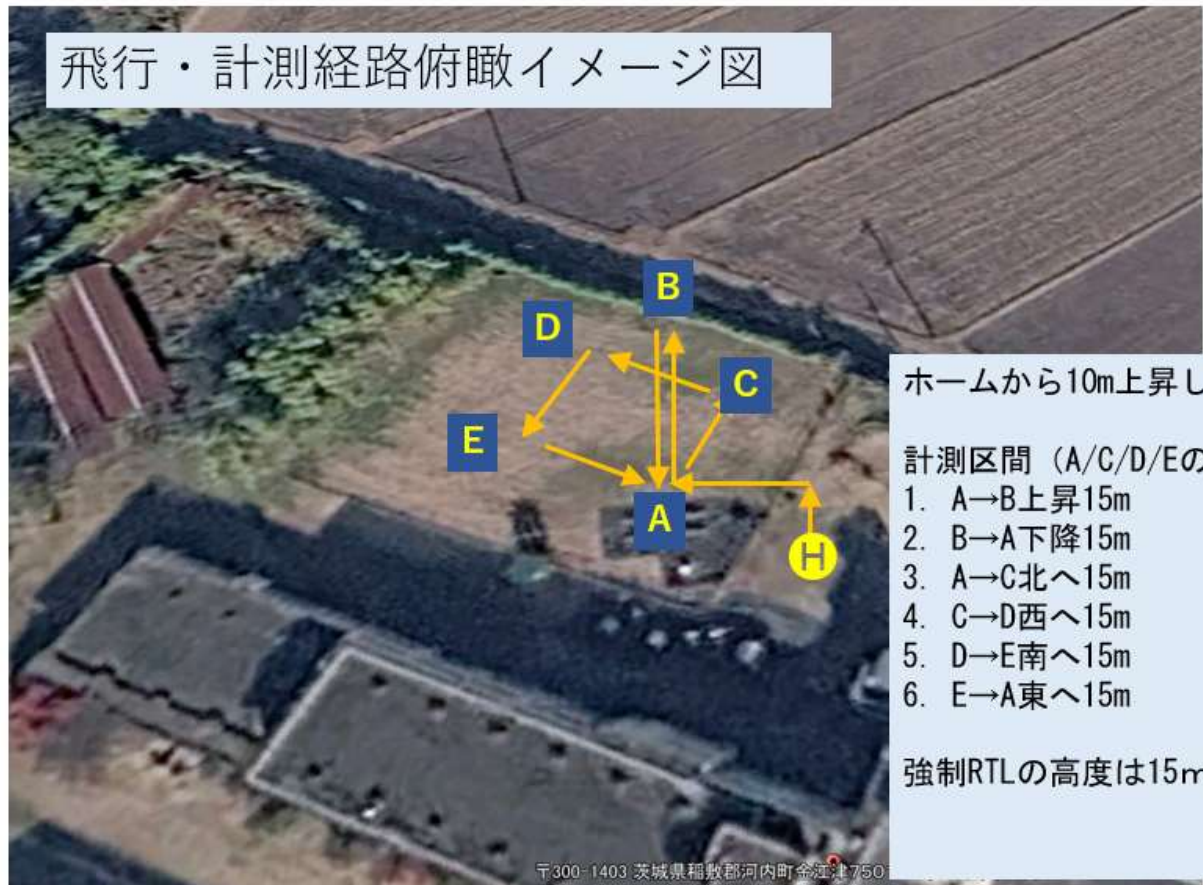
※専用に作成したWayPointファイルが必要。機体の向きは、常に北向きで飛行するようにしている

前提:

- ・計測開始、終了のトリガは、サーボ9番のPWM信号を1900と1100に変化させることで本プログラム側から検知する。よって、サーボ9番が未使用チャンネルであること。使用済の場合、他の未使用チャンネルに変更し、WayPointファイルのCMD:183行の値とそろえる
- ・専用に作成したWayPoint が Mission Planner等で機体にあらかじめ書き込まれていること
- ・ホームポイントを参照するため、実行環境と同一のカレントディレクトリに .waypoints ファイルが存在すること
- ・GPSフィックス、EKF、コンパス等の事前チェックに問題がないこと

安全対策

- ・ARM前にバッテリー残量が20%未満の場合、ARM/離陸せずに終了する
- ・バッテリー残量が20%未満になった場合、強制的にRTLする（RTL_ALTITUDE[15m]まで上昇して帰還）
- ・電圧ベースの保険（残量%が取得できない機体への保険）として、起動時にオペレーターが手動でセル数を入力し、LOW_CELL_VOLTAGE[V/セル]を下回った場合に強制RTLする



<使用するファイル2点と実行上の注意点>

1. プログラムファイル：[measure_battery_auto.py](#)

1-1.接続先 connect 部分の修正

MissionPlanner：VSC等実行環境のPCアドレスに修正する

実機：TBD

1-2.サーボの空きチャンネルへの変更

MissionPlanner：不要

実機：TBD

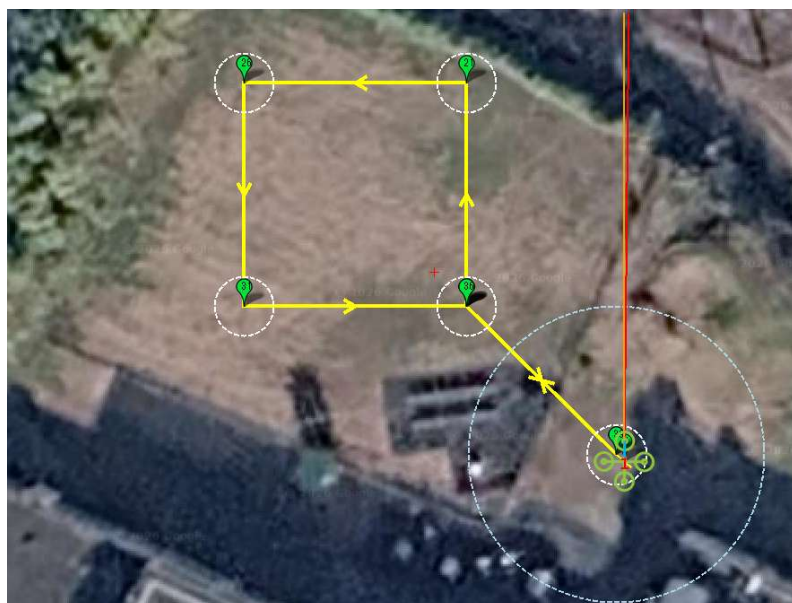
2. ウェイポイントファイル：[Mission-TennisCourt.waypoints](#)

pythonスクリプト実行環境カレントフォルダに配置する

事前に実行環境（MissionPlanner/実機への書き込みが必要）

MissionPlannerでの実行手順

1. MissionPlannerを起動し、シミュレーションメニューからMultiRotorを選択
2. フライトプランメニュー、上記ウェイポイントファイルを読み込む
3. WPの書き込みをする



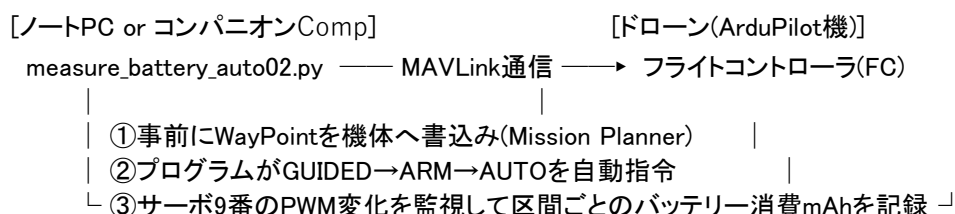
4. WSL/VSC 環境から `measure_battery_auto` を実行
メッセージに従ってOKであれば、Enterキー/Spaceキーで実行を進める

ここから先、要ご相談

=====

実機フライトまでの準備手順

全体像(このシステムは何をするのか)



ポイントは3つです。

1. WayPointファイルが、飛行ルートと「計測トリガ(サーボ9番のON/OFF)」の両方を担っている
2. Pythonプログラムが、離陸～着陸の操作を自動で送り、サーボ信号を見て消費電力を記録する
3. 両者はサーボ9番という約束事で連携している(ここがズレると計測できない)

STEP 0 – 準備するもの

項目	内容
機体	ArduPilot(Copter)が載った、飛行調整済みのドローン
地上局PC	Mission Planner が動くPC (WayPoint書込み用)
実行環境	Python + dronekit が入ったPC or 機上コンピュータ(Raspberry Pi等)
通信手段	テレメトリ無線(SiK 57600bps) または 機上コンピュータからのUSB/UART/TCP接続
バッテリー	セル数を把握しておく(3S/4Sなど)
場所	上空25～40mまで上昇するため、法規・許可・十分に広い安全な空域

このミッションは**高度25mまで上昇**し、四角形に自動飛行します。飛行させる前に飛行許可・承認、周囲の安全確認が必須です。

STEP 1 – WayPointファイルの中身を理解する

プログラムのコメントによると、WayPointファイルは以下の構造になっています(機体は常に北向き)。

区間	内容
先頭	NAV_TAKEOFF (10mへ自動離陸) → 10mで5秒待機
区間1	上昇 10m → 25m
区間2	下降 25m → 10m
区間3～6	南・西・北・東の四角形飛行(各辺)
最後	ホーム上空へ帰還 → LOITER_UNLIM(ホバリング待機、自動着陸しない)

最重要ポイント:計測トリガ 各区間の開始点と終了点に DO_SET_SERVO(コマンド番号183)が入っています。

- ・ 区間開始 → サーボ9番を **PWM 1900** にする
- ・ 区間終了 → サーボ9番を **PWM 1100** にする

PythonプログラムはこのPWMが1500を超えた瞬間＝計測開始／下回った瞬間＝計測終了として、区間ごとの消費電力(mAh)を切り出します。

確認事項:WayPoint内の DO_SET_SERVO の「サーボ番号」が **9** になっていること。プログラム側の設定 ([measure_battery_auto02.py:54](#) の SERVO_CHANNEL = 9) と必ず一致させてください。ここが唯一の連携ポイントです。

STEP 2 – 機体側パラメータの確認・設定(Mission Planner)

Mission Planner の Config → Full Parameter List で以下を確認します。

① サーボ9番を「未使用」にする

- ・ SERVO9_FUNCTION = 0(Disabled / 未使用)にしておく
 - もし9番が既に他用途(カメラ等)で使われているなら、空いている別チャンネルに変更し、
 - WayPointの DO_SET_SERVO の番号
 - プログラムの SERVO_CHANNEL の両方をその番号に揃えてください。

② AUTO_OPTIONS(自動離陸の許可)

通常ArduPilotは「AUTOモードでもRC送信機のスロットルを上げないと離陸しない」安全設計です。PC(MAVLink)だけで完全自動飛行するにはこれを解除します。

- ・ AUTO_OPTIONS = 3(bit0=AUTO中のARM許可 + bit1=スロットル操作なしで離陸開始)
このプログラムは起動時に自動で AUTO_OPTIONS を設定してくれます([measure_battery_auto02.py:84](#) の ensure_auto_options)が、事前に確認しておく心安心です。

③ バッテリーモニタ(消費電力mAhの取得に必須)

- ・ BATT_MONITOR = 4(Voltage and Current) 等、電流センサ付きで設定・キャリブレーション済みであること
- ・ これが無いと消費mAh(current_consumed)や残量%が取得できず、計測の主目的が達成できません

④ フェイルセーフ(安全網)

- バッテリー、RC、GCSのフェイルセーフを適切に設定
- RTL時の高度 RTL_ALT を安全な値に

STEP 3 – WayPointを機体へ書き込む(Mission Planner)

1. Mission Planner を機体に接続
2. PLAN 画面を開く
3. File → Load WP File で **Square11.waypoints** を読み込む
4. 地図上でルートを目視確認(離陸位置＝ホームが安全な場所か、40m上昇や四角形が空域内に収まるか)
5. Write WPs (Upload) で機体へ書き込む
6. 一度 Read WPs で読み戻し、意図通り入っているか確認
WayPointの緯度経度は**作成した場所の座標**です。別の飛行場で飛ばす場合は、その場所に合わせ作り直してください(そうしないと想定外の場所へ飛ばそうとします)

STEP 4 – 通信接続の設定(プログラム側)

[measure_battery_auto02.py:50](#) の接続文字列を、**実機の接続方法**に合わせて書き換えます。

CONNECTION_STRING = "tcp:192.168.3.210:5762" # ← 現在はシミュレータ/ネットワーク向け
実機での代表例:

接続方法	設定例
テレメトリ無線(SiK)をPCのUSBに挿す	"/dev/ttyUSB0" + BAUD = 57600 を有効化
機上コンピュータからUART直結	"/dev/serial0" 等 + 適切なBAUD
ネットワーク経由(mavlink-router等)	"udp:127.0.0.1:14550" など

シリアル接続にする場合は、コメントアウトされている BAUD = 57600 ([51行目](#)) を有効に

し、connect(CONNECTION_STRING, baud=BAUD, wait_ready=True) のようにボーレートを渡してください。

Mission Planner とこのプログラムが**同じ1本のテレメトリ**を同時に奪い合うと接続できないことがあります。両方から見たい場合は MAVProxy / mavlink-router で信号を分岐させます。まずはプログラム単独で接続確認するのが簡単です。

STEP 5 – 実行環境の準備

プログラムを動かすPC/機上コンピュータで:

```
pip install dronekit
```

環境によっては pymavlink も

動作確認は、いきなり実機ではなく SITL(シミュレータ) で一度通しておく心安心です。

STEP 6 – 現地での事前チェック(飛ばす直前)

1. プロペラを外した状態でまず通信・アーム挙動を確認するのが理想
2. 機体を安全な離陸位置(＝ホームになる場所)に設置
3. 電源投入 → GPSフィックス(3D Fix)を待つ
4. Mission Planner で以下を確認
 - GPS: 3D Fix、HDOP低い、衛星数十分
 - EKF: 緑(正常)
 - コンパス/加速度キャリブ済み、振動・磁気異常なし
 - バッテリー電圧・残量が表示されている
5. プロペラ装着、周囲の人・障害物なし、安全距離を確保
プログラム側も起動時に is_armable(GPS/EKF/コンパス等)を自動チェックし、OKになるまで待機します([measure_battery_auto02.py:75](#))。

STEP 7 – プログラム実行 → 飛行

```
python homework2/measure_battery_auto02.py
```

実行後の流れ(プログラムが誘導してくれます):

1. **接続** →「Connected.」表示
2. **バッテリーセル数を入力**(3Sなら 3 + Enter、space/Enterだけで既定3S)
 - これは残量%が取れない機体向けの「電圧ベース保険」の下限計算に使われます(3.4V/セル×セル数を下回ると強制RTL)
3. **アーム可能待機** → 自動でGUIDEDモード&ARM
4. **離陸確認プロンプト**(ビーブ音が鳴ります)
 - space または Enter → 離陸実行
 - 他の文字+Enter → キャンセルしてディスアーム
5. **AUTOモードへ切替** → ミッション開始(自動離陸→上昇→下降→四角形飛行→ホーム上空)
6. 飛行中、区間ごとに消費mAhをコンソール表示 & CSVに都度保存
7. 全6区間完了 → ホーム上空でホバリング待機
8. **着陸確認プロンプト**
 - space/Enter → LANDモードで自動着陸 → ディスアーム検知で終了

安全機能(飛行中も自動作動)

- バッテリー残量 **20%未満** で強制RTL
- 入力セル数 × **3.4V/セル** を **5秒連続**で下回ったら強制RTL
- Ctrl+C で中断してもそこまでの結果はCSV保存されます

必ず送信機を手に持ち、いつでも手動介入(モード切替)できる体制で行ってください。自動離陸を許可しているため、ARMした時点でプロペラが回り、そのまま離陸します。

STEP 8 — 結果の確認

実行フォルダに battery_consumed_YYYYMMDD_HHMM.csv が生成され、区間ごとに以下が記録されます。
leg(区間番号), elapsed_sec(経過秒), consumed_mah(消費mAh),
start/end_voltage(電圧), start/end_temp_c(温度), 開始/終了の緯度経度

つまずきやすいポイント まとめ

症状	原因・対処
区間計測が始まらない/0区間のまま	WayPointのDO_SET_SERVO番号とプログラムのSERVO_CHANNEL(9)が不一致。またはSERVO9_FUNCTIONが0でない
ARM・AUTOになるが離陸しない	AUTO_OPTIONSが未設定(bit0+bit1=3が必要)
consumed_mahが空欄	BATT_MONITORが電流センサ付きになっていない/キャリブ未実施
接続できない	CONNECTION_STRING/BAUDが実機と不一致、またはMission Plannerと競合
想定外の場所へ飛ぼうとする	WayPointの座標が現地と違う(現地で作り直す)

このシステムの心臓部は「WayPointのサーボ信号 ⇄ プログラムのサーボ監視」の連携です。ここだけは番号を必ず揃える、という点を最優先で覚えておけば、あとは画面の指示通りに進められます。

WayPointファイル(Square11.waypoints)が手元にあれば、中身(特にDO_SET_SERVOの行と座標)と一緒に確認できます。共有いただければ、より具体的にチェックできます。